

Scaling Manual-Grounded Appliance Manipulation with Data Synthesis and Unified Planning

Anonymous Author(s)

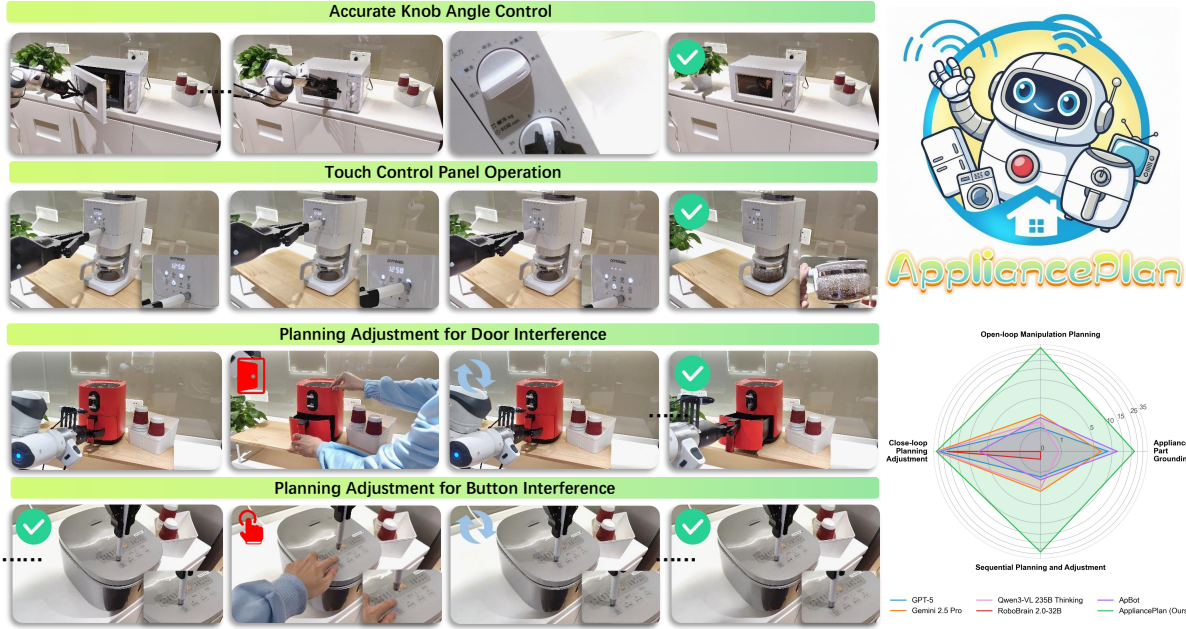


Figure 1: Capabilities of our AppliancePlan model. Left: AppliancePlan plans manipulation for diverse appliances and updates the plan when interrupted. Right: AppliancePlan outperforms strong large-model baselines on RealAppliance-Bench [9].

Abstract

Operating household appliances requires long-horizon planning that is state-dependent and robust to disturbances, yet existing large models fall short, as no sufficiently diverse, task-oriented dataset exists to support such planning. To bridge this gap, we propose MAGE, a scalable data synthesis pipeline that introduces a novel Hierarchical Appliance Graph (HAG) to automatically generate part grounding, long-horizon planning, and closed-loop recovery data from appliance manuals. With MAGE, we build UseAppliance, the first large-scale dataset for manual-grounded appliance manipulation planning, spanning 22 appliance categories with 89K+ part annotations, 53K+ manipulation tasks, and 33K+ closed-loop adjustment steps. Built on UseAppliance, we develop AppliancePlan, an end-to-end model for manual-grounded appliance manipulation planning. On RealAppliance-Bench, AppliancePlan with only 7B parameters achieves over 10× the best baseline on open-loop planning and consistently outperforms state-of-the-art models across all tasks. Real-robot experiments on six household appliances further confirm effective sim-to-real transfer, marking an important step toward general-purpose household robotics. The full pipeline, dataset, and model will be open-sourced upon paper acceptance.

CCS Concepts

• Computing methodologies → Robotic planning; Vision for robotics; • Computer systems organization → Robotics.

Keywords

Appliance manipulation planning, household robotics, multimodal foundation models

1 Introduction

Enabling robots to operate household appliances is a key prerequisite for practical home autonomy. Appliance manipulation is substantially harder than generic pick-and-place: appliances vary widely in interface layout, operation logic, and state transitions, and successful execution requires long-horizon, state-dependent procedures that must remain robust to disturbances.

Humans address this by consulting manuals—an approach that naturally decomposes the task into three sub-capabilities: appliance part grounding, open-loop manipulation planning, and closed-loop planning adjustment. RealAppliance-Bench [9] provides a unified evaluation protocol for these capabilities, pairing 100 real appliance manuals with high-fidelity digital assets. Yet existing MLLMs and embodied planning models still perform poorly on RealAppliance-Bench. Prior methods either focus on simple interactions (e.g., pressing a single button) [4, 14, 20] or rely on multi-stage zero-shot pipelines built upon proprietary models, incurring high inference latency and compounding errors [17, 20, 30]. **The fundamental bottleneck is the lack of large-scale, high-quality training data tailored to manual-grounded appliance manipulation.**

To address this bottleneck, we introduce MAGE (Manual-grounded Appliance data GEneration pipeline), a scalable data synthesis

pipeline built on a novel Hierarchical Appliance Graph (HAG). HAG represents each appliance manual as a four-level hierarchy—from document and page structure down to individual parts and state-transition graphs. MAGE collects and classifies manuals via autonomous agents to form the upper layers of each HAG, then completes the lower layers through model-assisted annotation. With the HAG in place, data generation reduces to combinatorial graph traversal—sampling state pairs for plan synthesis and injecting disturbances for closed-loop recovery—with human verification at every stage. This yields three data types: (i) multi-view bounding-box for part grounding; (ii) long-horizon manipulation plans with systematic state-space coverage; and (iii) planning-aligned observation images with closed-loop recovery data.

With MAGE, we build UseAppliance, the first large-scale dataset for manual-grounded appliance manipulation planning, with four key properties: (i) *high quality*—Core annotations are human-labeled and double-verified; (ii) *rich diversity*—spanning 22 appliance categories from manufacturers worldwide with diverse control interfaces and operation procedures; (iii) *multi-task coverage*—providing unified supervision for all three capabilities with 89K+ part bounding boxes, 53K+ manipulation tasks, and 33K+ closed-loop adjustment steps; and (iv) *extensibility*—the MAGE pipeline is fully decoupled from specific appliances, allowing new training data to be generated from any appliance manual.

Furthermore, we propose AppliancePlan, a unified end-to-end model for manual-based appliance manipulation planning. Beyond the three main planning objectives, AppliancePlan incorporates three complementary auxiliary tasks—bidirectional manual-part alignment, key-step action prediction, and part state judgment—that provide targeted supervision for spatial grounding, parameter precision, and state-aware plan revision, respectively. On RealAppliance-Bench, AppliancePlan with only 7B parameters consistently outperforms proprietary MLLMs such as GPT-5 and embodied planning baselines across all tasks: it achieves over 10× the best baseline on open-loop planning (31.36% vs. 2.68% task success) and leads the most realistic sequential setting by a wide margin (28.07% vs. 4.08%). As shown in Figure 1, real-robot experiments on six household appliances further confirm effective physical transfer, with 40.00% task success compared to 3.33% for GPT-5.

In this work, our main contributions are:

- **MAGE**, a novel scalable data synthesis pipeline based on a Hierarchical Appliance Graph (HAG) representation, which automatically generates part grounding, long-horizon planning, and closed-loop recovery data at scale from appliance manuals with strategic human verification.
- **UseAppliance**, the first large-scale dataset for manual-grounded appliance manipulation planning, spanning 22 appliance categories with 89K+ part annotations, 53K+ appliance manipulation tasks, and 33K+ closed-loop adjustment steps.
- **AppliancePlan**, the first end-to-end model for long-horizon appliance manipulation planning, surpassing all baselines on every RealAppliance-Bench track—notably over 10× on open-loop planning (31.36% vs. 2.68% task success). Real-robot experiments further confirm substantial gains (40.00% vs. 3.33% for GPT-5).

2 Related Work

2.1 Appliance Manipulation

Enabling robots to operate household appliances has been explored along two directions. The first targets direct interaction with household devices [4, 14, 20], but mainly handles short-horizon behaviors (e.g., opening a microwave door) without manual-grounded operation. The second direction explicitly uses manuals for action planning. CheckManual [17] and ApBot [30] show the value of manual information, but both rely on multi-stage zero-shot pipelines with cascaded external modules, incurring high latency and compounding errors. ApBot’s action space is further limited to direct state transitions and incremental adjustments, precluding single-step prediction of continuous parameters such as rotation angles. Our work addresses this gap with large-scale training data containing fully parameterized atomic actions and a unified end-to-end model.

2.2 Large Models in Embodied Planning

Multimodal foundation models [2, 13, 15, 18, 22] have greatly improved long-context understanding and multimodal reasoning, spurring embodied planning research. On the action execution side, SayCan [1] combines language models with skill affordances for grounded decision making, PaLM-E [7] integrates multimodal perception and language modeling in a single embodied model, RT-2 [5] casts robot actions as text tokens to enable transfer from web-scale data, and OpenVLA [12] makes large-scale VLA training broadly accessible. On the planning side, RoboBrain [11] and RoboBrain 2.0 [25] leverage embodied planning data [6] to improve long-horizon plan generation with stronger spatial and temporal reasoning. However, these models are evaluated primarily on generic object manipulation and fall short on manual-grounded appliance operation. The fundamental bottleneck remains the lack of large-scale, high-quality training data tailored to these capabilities.

3 MAGE: Data Synthesis Pipeline

The fundamental bottleneck for manual-grounded appliance manipulation is the lack of large-scale training data. We introduce **MAGE** (Manual-grounded Appliance data **GE**neration pipeline), a scalable data synthesis pipeline built on a novel Hierarchical Appliance Graph (HAG). MAGE automates both HAG construction and data generation: the first two stages (Secs. 3.2–3.3) build each HAG with automated collection agents and human-verified annotation, while the latter two (Secs. 3.4–3.5) consume the completed HAG to synthesize appliance manipulation planning data with human verification gates after each stage (Figure 2).

3.1 Hierarchical Appliance Graph Formulation

We formalize appliance manual knowledge as a **Hierarchical Appliance Graph** (HAG) $\mathcal{G}_m = (\mathcal{V}, \mathcal{E})$, where nodes $\mathcal{V}$ represent semantic entities at four granularity levels and edges $\mathcal{E}$ encode containment, reference, and state-transition relations.

A HAG organizes knowledge into four layers: (i) *Document layer*: the manual root v_m representing the entire document; (ii) *Page layer*: typed page nodes $v_p^{(k)}$ connected to the root by containment edges, such as product-overview and operating-instructions pages; (iii) *Part layer*: part nodes $v_c^{(j)}$ linked to their describing pages, each

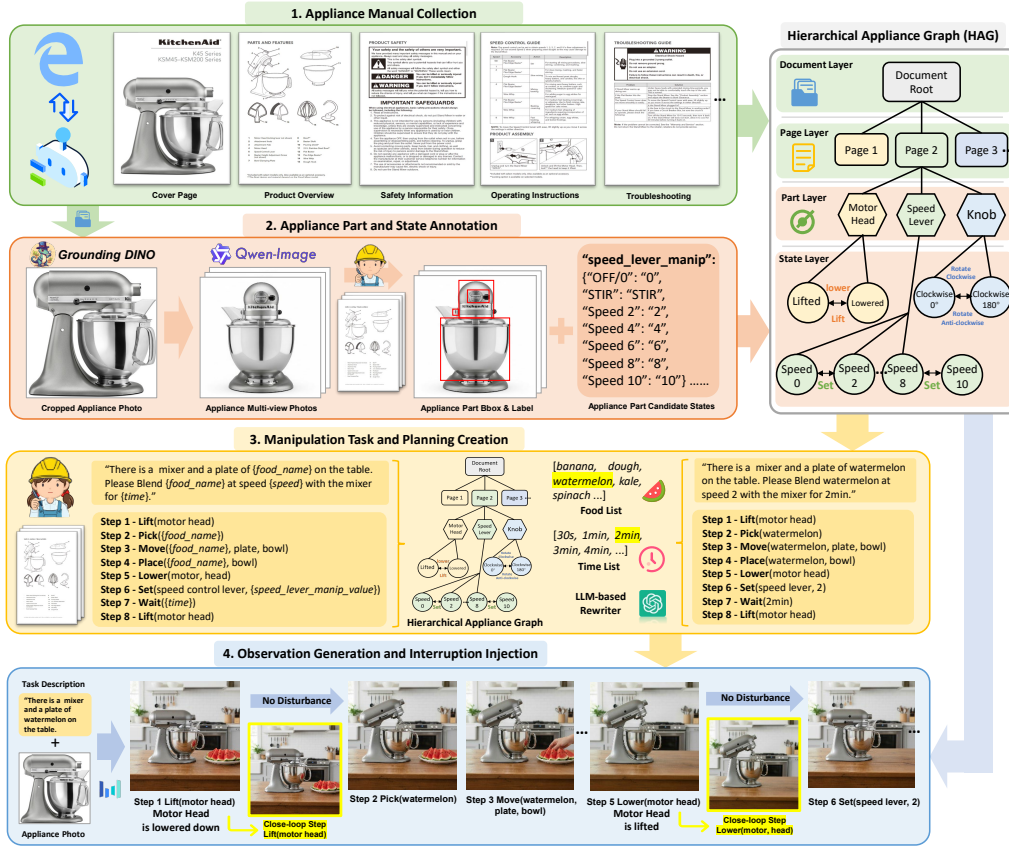


Figure 2: Overview of MAGE. The first two stages (Secs. 3.2–3.3) construct the Hierarchical Appliance Graph (HAG) via automated collection and human-verified annotation; the latter two (Secs. 3.4–3.5) consume the completed HAG to synthesize three types of data—part grounding annotations, open-loop manipulation plans, and observation images with closed-loop recovery data.

annotated with a visual bounding box in the corresponding appliance view; (iv) *State layer*: state nodes $v_s^{(i)}$ representing discrete or continuous part configurations, such as knob angles and button states. Directed action edges $v_s \xrightarrow{a} v_{s'}$ encode which atomic action a transitions the part from state s to s' .

This formulation provides three useful properties: (1) **Combinatorial task coverage**—sampling initial–goal state pairs yields diverse tasks; (2) **Locally verifiable annotations**—each layer can be independently audited; (3) **Domain-agnostic extensibility**—the four-layer schema generalizes to any product manual.

3.2 Appliance Manual Collection

This stage builds the document and page layers of each HAG. We design a VLM-powered **manual collection agent** that operates in a ReAct-style loop [29] to autonomously search manufacturer websites and manual databases, download candidate manuals, and filter out low-quality ones based on page count, file size, and resolution, each instantiated as a document-layer node v_m . The agent then classifies each page into one of nine semantic types (product overview, operating instructions, etc.) to populate the page layer.

3.3 Appliance Part and State Annotation

This stage completes the part and state layers of each HAG.

Part Grounding Annotation. Since manual diagrams and real appliance appearances differ substantially, we construct multi-view observation images to bridge this domain gap. For manuals with clear cover photos, we use an open-vocabulary object detector [16] with the appliance category as the prompt to detect the appliance region; if a high-confidence bounding box is returned, we crop it from the cover and then synthesize three canonical views (front, oblique front-side, and oblique top-down) [28]. Human annotators label visible parts with bounding boxes on these views guided by product-overview pages, further enriched with spatial augmentation.

State & Transition Annotation. For each part node $v_c^{(j)}$, we assign a *state-type label* and construct a directed state-transition graph within the state layer. A VLM reads the relevant manual pages and proposes candidate states together with inter-state transitions; human annotators verify and correct the proposals against the manual. All parts are unified into discrete state graphs: binary parts (e.g., doors) yield two-node graphs with complementary action edges, discrete parts (e.g., mode selectors) are enumerated by mode with button-press edges, and continuous parts (e.g., knobs) are discretized at manual-specified granularity with rotation edges. Edges may be unidirectional when the manual specifies irreversible

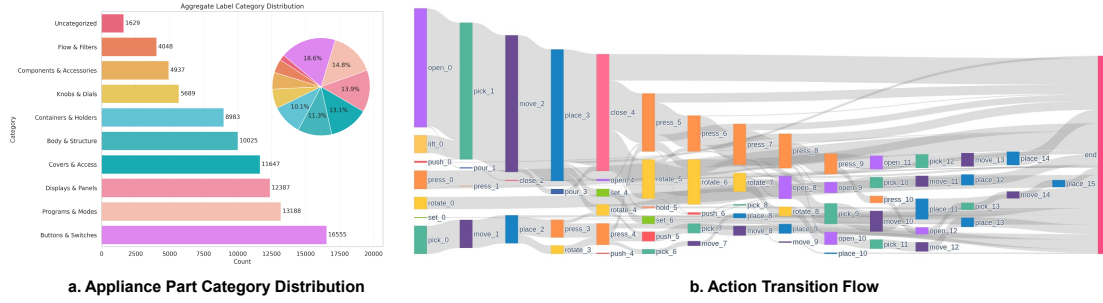


Figure 3: Statistics of UseAppliance. (a) Long-tail distribution of 89K+ part bounding boxes. (b) Action-transition flow of 53K+ open-loop planning tasks.

operations. The resulting state nodes $\{v_s^{(i)}\}$ and directed action edges $v_s \xrightarrow{a} v_{s'}$ complete the HAG for downstream plan synthesis.

3.4 Manipulation Task and Planning Creation

This stage consumes the completed HAG to synthesize open-loop task-plan pairs, producing long-horizon manipulation planning data. The key idea is to treat task generation as *structured graph traversal*: each task corresponds to a path in the state-transition graph, and plan generation is driven by traversing the state-transition graph and instantiating templates with sampled parameters (Figure 2).

Task and Plan Templates. For each appliance, we manually design a set of task and plan templates based on its manual and the corresponding HAG. Task templates contain typed placeholders for the target state, duration, and interacting objects (e.g., “Heat the {object} for {duration} at {target_state}”). Plan templates are ordered sequences of atomic actions drawn from the action vocabulary defined in RealAppliance-Bench, including appliance manipulation (Press, Rotate, etc.), object manipulation (Pick, Place, etc.), and timing (Wait). Every appliance action specifies the target part and an operation placeholder (rotation angle, press count, etc.); every object action specifies the target object and relevant locations; Wait only takes a duration parameter. Task and plan templates are paired: each task template maps to a corresponding plan template that realizes the described goal, with shared parameters kept consistent.

Template-based Generation. Given a task-plan template pair, we generate concrete instances by sampling an initial-goal state pair (s_0, s^*) from the HAG state layer and drawing task attributes (interacting objects, durations, etc.) from appliance-specific lists. These jointly fill the task template placeholders. The plan template already defines the action sequence skeleton; we fill operation placeholders from the state difference between s_0 and s^* together with the sampled task attributes, yielding an open-loop plan:

$$a_t = \mathcal{T}_t^{\text{plan}}(\Delta(s_0, s^*), \mathcal{A}), \quad Y_0 = (a_1, a_2, \dots, a_T), \quad (1)$$

where $\mathcal{T}_t^{\text{plan}}$ denotes the t -th step of the plan template, $\Delta(s_0, s^*)$ encodes the state difference (rotation angle $\alpha^* - \alpha_0$, discrete step count, etc.), and $\mathcal{A}$ denotes the sampled task attributes (objects, durations, etc.). An LLM rewrites the filled task templates into diverse natural-language instructions while preserving semantic fidelity. Uniform sampling of (s_0, s^*) pairs ensures systematic state-space coverage, avoiding the bias of human demonstrations toward common configurations.

3.5 Observation Generation and Interruption Injection

This stage generates planning-aligned observation images and injects structured disturbances to produce closed-loop recovery data.

Visual State Prediction. We use Seedream 4.0 [24] as the generative world model $\mathcal{M}_{\text{gen}}$ to iteratively generate observation images. Given the appliance photo and task description, we first generate the initial observation I_0 ; at each subsequent step t we convert the action a_t into a textual prompt describing the expected visual change and generate:

$$I_{t+1} = \mathcal{M}_{\text{gen}}(I_t, \text{prompt}(a_t)), \quad (2)$$

producing an observation sequence $(I_0, I_1, \dots, I_T)$ aligned with the action sequence. We apply cross-step consistency checking and human verification to mitigate visual drift.

Closed-loop Interruption Injection. To create closed-loop adjustment supervision, we inject hypothetical disturbances into intermediate steps of the generated observation sequences. We define three categories of interruptions aligned with the three part types: (i) *binary-state errors*—a part unexpectedly changes state; (ii) *button mis-touches*—an incorrect number of presses is executed; and (iii) *rotation deviations*—a knob is turned to an incorrect angle.

For each injected interruption at step t , we generate a post-disturbance observation image I'_t using the generative world model and annotate the corrective action a_{t+1}^{corr} conditioned on the task context and disturbed observation. This yields closed-loop training triples $(I'_t, \text{context}, a_{t+1}^{\text{corr}})$ that teach the model to detect deviations and generate appropriate recovery actions.

4 UseAppliance Dataset

Applying the MAGE pipeline to 22 appliance categories, we build UseAppliance, the first large-scale dataset for manual-grounded appliance manipulation planning.

4.1 Key Characteristics

As the first large-scale training dataset for manual-grounded appliance manipulation planning, UseAppliance offers four key characteristics:

- **High quality**—All part bounding boxes and state-transition graphs are human-labeled; task and plan templates are manually authored per appliance; every annotation passes at least one round of independent human verification.

- *Multi-task coverage*—The dataset provides unified supervision for part grounding with 89K+ bounding boxes, open-loop planning with 53K+ task-plan pairs, and closed-loop adjustment with 33K+ recovery steps, enabling joint training of all three capabilities.
- *Rich diversity*—Spanning 22 appliance categories from manufacturers worldwide, UseAppliance covers a wide range of control interfaces such as knobs, buttons, touch panels, and levers, as well as diverse operation procedures including heating, blending, and brewing.
- *Extensibility*—Since the MAGE pipeline takes only an appliance manual as input, new appliance categories can be onboarded without modifying the pipeline itself.

4.2 Data Statistics

Quantitative statistics are summarized in Table 1. As shown in Figure 3(a), part annotations follow a long-tail distribution: common parts (e.g., doors, buttons, knobs) receive dense annotations while appliance-specific controls appear infrequently, encouraging robustness to rare structures. Figure 3(b) visualizes the action-transition flow across appliance categories: tasks average 8.15 open-loop steps, with action distributions varying significantly by category—e.g., knob-heavy appliances are dominated by Rotate, while panel-based appliances rely more on Press—confirming that the dataset captures diverse operation patterns.

Table 1: Summary statistics of UseAppliance.

Metric	Value
Total # Appliance manuals	2,120
Total # appliance manual pages	50,727
Total # Appliance parts	1,584
Total # Part bounding boxes	89,088
Total # Appliance manipulation tasks	53,854
Average # Open-loop manipulation steps	8.15
Median # Open-loop manipulation steps	8
Total # Close-loop manipulation steps	33,815

5 AppliancePlan Model

5.1 Model Architecture

AppliancePlan is built on Qwen2.5-VL-7B-Instruct [3]. Given visual inputs $\mathcal{I} = \{I_1, \dots, I_t\}$ (manual pages and observation images) and a textual task instruction L , the vision encoder and vision-language merger project the inputs into the LLM token space, and the language model predicts the output autoregressively:

$$P_\theta(Y | H_v, X) = \prod_{n=1}^N P_\theta(y_n | y_{<n}, H_v, X). \quad (3)$$

where H_v denotes the projected visual tokens, $X = \text{Tok}(L)$ the text tokens, N the output sequence length, and $Y = (y_1, \dots, y_N)$ the output sequence.

5.2 Model Training

AppliancePlan is trained with 3 main objectives and 3 auxiliary objectives constructed from UseAppliance. All objectives are formulated as next-token prediction and optimized with cross-entropy loss.

5.2.1 Main Planning Objectives. The three main objectives correspond to the three core capabilities. Let $\mathcal{D}_g$, $\mathcal{D}_o$, and $\mathcal{D}_c$ denote the training splits for part grounding, open-loop planning, and closed-loop adjustment, respectively.

Appliance Part Grounding. This objective bridges the domain gap between symbolic part descriptions in manuals and their pixel-level appearance in observation images. Given a multimodal input $\mathbf{u}_g = (\mathcal{I}_{\text{manual}}, I_{\text{obs}}, L_{\text{part}})$, the model predicts the bounding-box coordinate sequence $\mathbf{b} = (x_1, y_1, x_2, y_2)$.

$$\mathcal{L}_g = \mathbb{E}_{(\mathbf{u}_g, \mathbf{b}) \sim \mathcal{D}_g} [-\log P_\theta(\mathbf{b} | \mathbf{u}_g)]. \quad (4)$$

Open-loop Manipulation Planning. While grounding localizes *where* to act, this objective trains the model to reason about *what* to do and in what order. From the manual pages $\mathcal{I}_{\text{manual}}$, an initial observation I_0 , a task instruction L_{task} , and the atomic action set $\mathcal{A}$, the model generates the full open-loop plan $Y_o = (a_1(\psi_1), \dots, a_T(\psi_T))$ as a single autoregressive sequence, with one parameterized action per line.

$$\mathcal{L}_o = \mathbb{E}_{(\mathbf{u}_o, Y_o) \sim \mathcal{D}_o} \left[-\sum_{n=1}^{|Y_o|} \log P_\theta(y_n^o | y_{<n}^o, \mathbf{u}_o) \right], \quad (5)$$

where $\mathbf{u}_o = (\mathcal{I}_{\text{manual}}, I_0, L_{\text{task}}, \mathcal{A})$ denotes the full multimodal input.

Closed-loop Planning Adjustment. This objective trains the model to predict corrective actions by comparing the expected plan with the actual execution state. At step t , the input comprises the current observation I_t , the task instruction L_{task} , the reference plan $P_{1:T}$, and the executed action history $A_{1:t}^{\text{exec}}$; the model predicts the next corrective action a_{t+1}^* :

$$\mathcal{L}_c = \mathbb{E}_{(\mathbf{u}_c, a_{t+1}^*) \sim \mathcal{D}_c} [-\log P_\theta(a_{t+1}^* | \mathbf{u}_c)], \quad (6)$$

where $\mathbf{u}_c = (I_t, L_{\text{task}}, P_{1:T}, A_{1:t}^{\text{exec}})$.

The overall main objective combines these three losses:

$$\mathcal{L}_{\text{main}} = \mathcal{L}_g + \mathcal{L}_o + \mathcal{L}_c. \quad (7)$$

5.2.2 Auxiliary Training Objectives. The three auxiliary objectives target complementary dimensions of the planning problem. Let $\mathcal{D}_{\text{align}}$, $\mathcal{D}_{\text{key}}$, and $\mathcal{D}_{\text{state}}$ denote the corresponding training splits.

Manual-Appliance Part Alignment. The main grounding objective only maps part names to locations. This auxiliary objective strengthens bidirectional part-manual correspondence with three complementary sub-objectives, all receiving manual pages and an observation image as input: (i) *Manual-side part identification*—the target part is highlighted by a bounding box $\mathbf{b}$ on the observation image; the model predicts the part name n^* ; (ii) *Observation-side part identification*—the target part is indicated by its center-point coordinate $\mathbf{c}$; the model predicts n^* ; (iii) *Part localization*—a part name L_{part} is provided; the model predicts the center-point coordinate $\mathbf{c}$ on the observation image. The alignment loss is:

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{\mathcal{D}_{\text{align}}} [-\log P_\theta(n^* | \mathbf{u}_{\text{mid}}) - \log P_\theta(n^* | \mathbf{u}_{\text{oid}}) - \log P_\theta(\mathbf{c} | \mathbf{u}_{\text{loc}})], \quad (8)$$

where $\mathbf{u}_{\text{mid}} = (\mathcal{I}_{\text{manual}}, I_{\text{obs}}, \mathbf{b})$, $\mathbf{u}_{\text{oid}} = (\mathcal{I}_{\text{manual}}, I_{\text{obs}}, \mathbf{c})$, and $\mathbf{u}_{\text{loc}} = (\mathcal{I}_{\text{manual}}, I_{\text{obs}}, L_{\text{part}})$.

Key-Step Action Prediction. This objective focuses supervision on steps involving precise action parameter prediction (e.g., rotation angles, button-press counts). We extract such steps from open-loop plans to form a key-step subset $\mathcal{D}_{\text{key}}$ and train the model with three

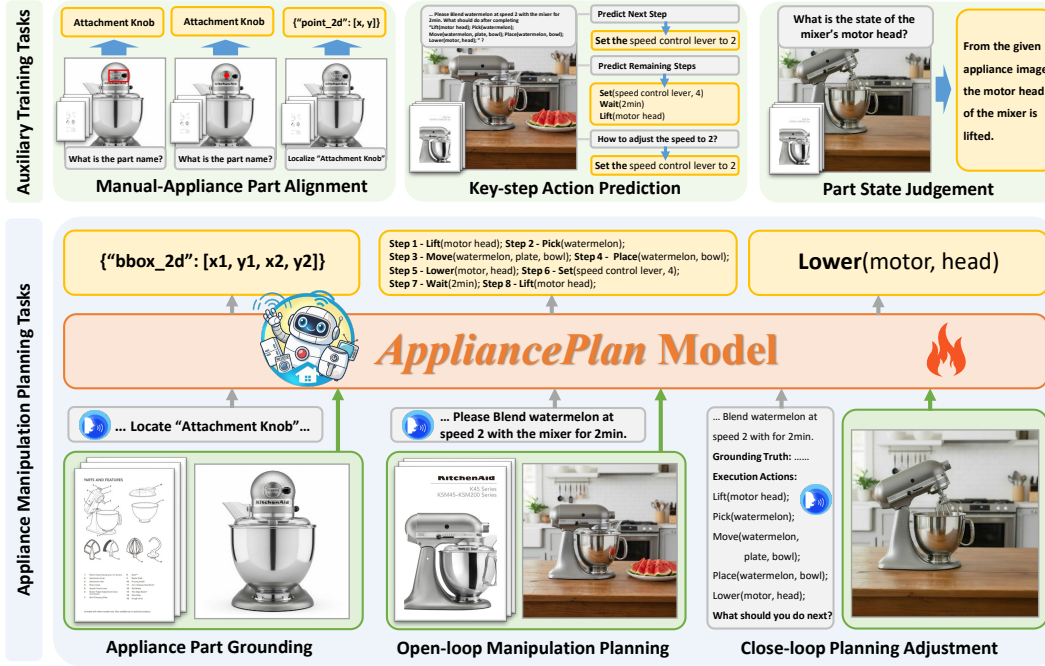


Figure 4: Overview of AppliancePlan model. AppliancePlan is an end-to-end model for manual-grounded appliance manipulation planning, trained with three main objectives and three auxiliary objectives.

complementary prediction targets: (i) *Next-step prediction*—from manual pages, the current observation, the task instruction, and the executed action history, the model predicts the next key action; (ii) *Remaining-step prediction*—with the same context, predict all remaining actions from the current step; (iii) *Parameter VQA*—the model answers a parameter question (e.g., target angle, press count) based on the current observation.

$$\mathcal{L}_{\text{key}} = \mathbb{E}_{(\mathbf{u}_{\text{key}}, a_{\text{key}}^*) \sim \mathcal{D}_{\text{key}}} \left[-\log P_{\theta} \left(a_{\text{key}}^* \mid \mathbf{u}_{\text{key}} \right) \right], \quad (9)$$

where $\mathbf{u}_{\text{key}} = (\mathcal{I}_{\text{manual}}, I_t, L_{\text{task}}, A_{1:t})$.

Part State Judgment. This objective provides explicit state-recognition supervision through VQA: the model receives manual pages, an observation image I_{obs} , and a question about a specific part’s state, and predicts the ground-truth state label s^* .

$$\mathcal{L}_{\text{state}} = \mathbb{E}_{(\mathbf{u}_{\text{state}}, s^*) \sim \mathcal{D}_{\text{state}}} \left[-\log P_{\theta} \left(s^* \mid \mathbf{u}_{\text{state}} \right) \right], \quad (10)$$

where $\mathbf{u}_{\text{state}} = (\mathcal{I}_{\text{manual}}, I_{\text{obs}}, L_{\text{state_query}})$ and $L_{\text{state_query}}$ is the state question.

The overall auxiliary objective is:

$$\mathcal{L}_{\text{aux}} = \mathcal{L}_{\text{align}} + \mathcal{L}_{\text{key}} + \mathcal{L}_{\text{state}}. \quad (11)$$

The full training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{main}} + \mathcal{L}_{\text{aux}}. \quad (12)$$

6 Experiments

6.1 Implementation Details

Training runs for 96 GPU-hours on NVIDIA H200 with a global batch size of 512, a learning rate of 1×10^{-5} with cosine decay, and a frozen vision encoder. We use DeepSpeed ZeRO-3 [23] with bfloat16 mixed precision. All loss weights are set to 1.0. During inference, we use a temperature of 0.1 for stable plan generation. Training

uses only UseAppliance, which has no overlap with RealAppliance-Bench, making all evaluations out-of-distribution.

6.2 Evaluation on RealAppliance-Bench

6.2.1 Evaluation Tracks and Metrics. RealAppliance-Bench [9] is a benchmark for manual-grounded appliance manipulation built on 100 high-fidelity digital appliance assets in NVIDIA Isaac Sim, spanning 14 common appliance categories. Each asset is aligned with its real-world manual in appearance, physical mechanisms, electronic mechanisms, and program logic. We consider the following four evaluation tracks:

- **Appliance Part Grounding:** Given relevant manual pages, observation, and part name, the model localizes the target appliance part by predicting its bounding box. We report average Intersection-over-Union (IoU) and mean Average Precision at IoU 0.5 (mAP@0.5).
- **Open-loop Manipulation Planning:** Given relevant manual pages, an initial observation image, a task instruction, and the available atomic action set, the model predicts the full manipulation sequence without feedback during execution. Following the benchmark protocol, we report *task completion rate* and *task success rate*.
- **Closed-loop Planning Adjustment:** Given relevant manual pages, a task instruction, a reference plan, the executed action history, the current observation image, and the available atomic action set, the model predicts the next corrective action under disturbances. Performance is measured by *step-wise success rate*.
- **Sequential Planning and Adjustment:** This evaluation-only setting combines the previous two tracks: the model first generates an open-loop plan from the manual, initial

Table 2: Results on RealAppliance-Bench across three standard evaluation tracks.

Baseline Model																	Total
Appliance Part Grounding (Average / mAP@0.5)																	
Proprietary MLLMs																	
GPT-5 [19]	12.71/7.69	4.84/0.00	12.00/8.51	16.33/17.85	13.23/9.67	28.83/28.57	20.24/11.94	10.42/4.34	32.97/33.33	10.01/9.67	4.72/1.33	19.17/18.75	11.17/9.67	4.80/0.00	12.15/8.59		
GPT-5 Mini [19]	9.08/7.46	1.85/0.00	7.18/3.19	9.71/0.00	4.43/3.22	10.83/0.00	13.95/14.92	2.45/0.00	19.56/0.00	1.82/0.00	2.48/0.00	7.59/6.25	5.33/0.00	4.94/0.00	6.51/3.49		
Gemini 2.5 Pro [10]	7.77/4.47	3.75/0.00	10.74/8.51	9.47/10.71	6.74/6.45	7.23/0.00	10.46/8.95	5.87/4.34	32.82/41.66	9.70/11.29	2.92/0.00	6.64/6.25	6.70/6.45	6.16/0.00	8.16/6.64		
Gemini 2.5 Flash [10]	8.21/4.47	2.46/0.00	7.94/7.44	12.14/17.85	2.28/3.22	16.67/0.00	11.22/7.46	2.18/0.00	21.16/16.66	6.06/4.83	1.52/1.33	5.06/0.00	4.81/6.45	8.38/0.00	6.67/5.06		
Open-source MLLMs																	
Qwen3-VL 8B Thinking [21]	5.55/1.49	0.25/0.00	2.14/1.06	2.02/0.00	1.11/0.00	5.29/0.00	6.09/0.00	3.93/0.00	5.25/0.00	2.52/1.61	0.40/0.00	0.00/0.00	3.03/3.22	9.93/0.00	2.92/0.69		
Qwen3-VL 8B Instruct [21]	2.27/0.00	0.00/0.00	0.44/0.00	0.63/0.00	0.15/0.00	0.15/0.00	2.83/0.00	1.56/0.00	4.58/0.00	0.85/0.00	0.32/0.00	0.00/0.00	1.46/0.00	8.32/0.00	1.32/0.00		
Qwen3-VL 235B Thinking [21]	5.19/2.98	0.99/0.00	3.41/3.19	1.01/0.00	1.96/0.00	0.25/0.00	6.78/0.00	1.58/0.00	6.01/0.00	1.08/0.00	0.12/0.00	0.00/0.00	1.50/0.00	8.64/0.00	2.80/0.87		
Qwen3-VL 235B Instruct [21]	4.58/4.47	0.00/0.00	0.61/0.00	0.61/0.00	1.15/0.00	0.46/0.00	4.64/0.00	1.61/0.00	6.28/0.00	0.86/0.00	0.00/0.00	0.00/0.00	0.44/0.00	7.35/0.00	1.75/0.52		
GLM-4.1V-Thinking [26]	3.88/1.49	0.00/0.00	0.54/0.00	0.98/0.00	1.67/0.00	0.42/0.00	4.01/0.00	1.55/0.00	4.71/0.00	0.93/0.00	0.00/0.00	0.00/0.00	1.33/0.00	11.63/0.00	1.74/0.17		
GLM-4.5V [26]	3.59/2.98	0.00/0.00	0.75/0.00	0.99/0.00	2.50/0.00	0.32/0.00	4.09/0.00	1.56/0.00	6.15/0.00	0.66/0.00	0.00/0.00	0.00/0.00	0.62/0.00	11.00/0.00	1.76/0.35		
Embodied Planning Models																	
RoboBrain 2.0-7B [25]	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00
RoboBrain 2.0-32B [25]	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00
ManualPlan [17]	3.56/0.00	0.00/0.00	1.01/0.00	2.46/0.00	2.37/0.00	3.08/0.00	5.61/0.00	1.25/0.00	2.79/0.00	0.20/0.00	0.15/0.00	1.66/0.00	1.73/0.00	5.07/0.00	1.92/0.00		
ApBot [30]	23.60/28.40	10.50/12.30	8.50/7.60	4.70/4.50	15.70/20.00	17.70/20.00	10.20/11.20	0.00/0.00	4.50/4.30	11.70/12.00	11.20/14.70	0.00/0.00	12.20/13.60	4.30/5.30	10.60/12.10		
AppliancePlan (Ours)	13.25/10.45	19.03/19.15	14.19/12.77	35.04/35.71	15.25/16.13	16.39/14.29	19.77/19.70	39.65/39.13	72.06/66.67	27.82/27.42	31.98/33.33	6.77/6.25	20.53/19.35	39.56/33.33	22.96/22.24		
- Manual-App. Part Alignment	17.74/11.94	5.77/2.13	13.42/10.64	18.09/21.43	22.43/19.35	25.78/14.29	23.75/21.21	19.56/21.74	52.06/58.33	18.46/14.52	26.93/32.00	16.63/18.75	14.04/6.45	32.73/25.00	19.27/17.34		
- Key-step Action Prediction	26.66/26.87	14.50/12.77	15.26/11.70	27.57/28.57	32.56/29.03	29.50/42.86	25.56/21.21	22.94/26.09	50.59/58.33	21.41/22.58	26.35/28.00	22.39/18.75	14.45/9.68	25.44/16.67	22.99/21.89		
- Part State Judgement	24.62/22.39	19.69/19.15	18.34/15.96	24.41/25.00	33.44/32.26	29.31/42.86	29.00/27.27	22.98/26.09	41.72/33.33	18.64/16.13	26.59/32.00	33.08/31.25	9.50/3.23	20.89/16.67	23.45/22.59		
Open-loop Manipulation Planning (Task Completion Rate / Task Success Rate)																	
Proprietary MLLMs																	
GPT-5 [19]	2.60/1.11	2.57/0.00	1.55/0.00	0.00/0.00	7.77/2.00	5.92/0.00	4.70/0.83	0.95/0.00	4.80/2.00	3.02/0.00	6.44/2.02	7.77/0.00	14.59/6.00	10.66/10.00	4.30/1.22		
GPT-5 Mini [19]	2.22/2.22	5.23/1.25	0.11/0.00	0.00/0.00	3.58/0.00	1.26/0.00	3.52/1.66	6.45/0.00	4.09/2.00	2.34/0.00	4.08/0.00	5.55/0.00	9.29/4.00	9.66/6.66	3.27/1.02		
Gemini 2.5 Pro [10]	0.59/0.00	3.47/1.25	3.71/0.33	0.00/0.00	3.18/0.00	2.01/0.00	4.51/4.16	0.47/0.00	0.50/0.00	1.07/0.00	10.72/8.08	8.88/0.00	15.80/12.00	8.00/6.66	4.08/2.45		
Gemini 2.5 Flash [10]	0.23/0.00	2.00/0.00	6.12/3.84	0.00/0.00	2.79/0.00	1.26/0.00	2.34/1.66	7.82/3.33	2.16/2.00	2.34/0.88	10.55/8.08	5.55/0.00	15.51/12.00	6.66/6.66	4.26/2.65		
Open-source MLLMs																	
Qwen3-VL 8B Thinking [21]	1.55/1.11	2.82/0.00	1.53/1.53	0.00/0.00	3.88/2.00	4.23/3.70	1.79/1.66	7.42/3.33	5.73/2.00	0.88/0.88	6.39/5.05	2.77/0.00	7.22/4.00	6.66/6.66	3.01/1.94		
Qwen3-VL 8B Instruct [21]	0.00/0.00	4.41/0.00	0.76/0.76	0.00/0.00	0.00/0.00	0.00/0.00	1.66/1.66	0.00/0.00	2.00/2.00	0.88/0.88	1.68/0.00	12.22/0.00	6.71/4.65	3.33/3.33	1.70/0.82		
Qwen3-VL 235B Thinking [21]	1.56/1.11	5.95/1.25	1.59/0.00	0.00/0.00	9.46/4.00	7.83/3.70	2.08/1.66	10.79/3.33	5.40/2.00	4.18/0.88	6.68/4.04	8.33/0.00	7.51/2.00	8.00/6.66	4.36/1.73		
Qwen3-VL 235B Instruct [21]	0.00/0.00	6.86/1.25	3.23/3.07	0.00/0.00	8.81/4.00	2.32/0.00	4.50/4.16	3.14/0.00	3.13/2.00	2.47/1.76	6.58/3.03	8.88/0.00	9.95/6.00	6.66/6.66	4.11/2.34		
GLM-4.1V-Thinking [26]	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00		
GLM-4.5V [26]	2.22/2.22	3.30/1.25	2.41/2.30	1.11/1.11	5.21/4.00	4.44/3.70	1.30/0.83	1.33/0.00	4.43/2.00	1.50/0.97	8.41/6.06	3.33/0.00	10.11/8.00	12.66/10.00	3.73/2.68		
Embodied Planning Models																	
RoboBrain 2.0-7B [25]	0.00/0.00	0.84/0.00	0.06/0.00	0.00/0.00	0.17/0.00	0.00/0.00	0.16/0.00	0.00/0.00	0.00/0.00	0.27/0.00	0.00/0.00	0.55/0.00	0.22/0.00	0.00/0.00	0.16/0.00		
RoboBrain 2.0-32B [25]	0.00/0.00	1.99/0.00	0.00/0.00	0.00/0.00	0.71/0.00	0.00/0.00	0.12/0.00	0.00/0.00	0.00/0.00	0.17/0.00	0.67/0.00	0.00/0.00	1.53/0.00	0.00/0.00	0.57/0.00		
ManualPlan [17]	6.12/0.00	8.76/0.00	2.003/0.00	0.92/0.00	3.20/0.00	5.75/0.00	11.27/0.00	10.44/0.00	6.44/0.00	4.51/1.76	2.20/0.00	11.32/0.00	11.50/3.99	2.17/0.00	5.61/0.40		
ApBot [30]	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	2.00/2.00	8.70/7.40	1.60/0.80	0.00/0.00	0.00/0.00	7.10/7.10	7.10/7.10	0.00/0.00	4.40/4.00	0.00/0.00	2.30/2.10		
AppliancePlan (Ours)	45.70/27.80	42.80/23.70	48.00/36.20	49.00/21.10	29.92/14.00	43.40/40.70	48.20/31.70	20.00/13.30	41.58/24.02	47.30/32.70	61.30/49.50	73.30/60.00	61.82/45.98	52.20/13.30	47.86/31.36		
- Manual-App. Part Alignment	45.50/28.90	34.30/16.30	45.00/34.60	48.70/24.40	36.68/15.98	63.40/55.60	36.60/15.80	28.70/6.70	43.22/23.98	53.10/41.60	52.80/41.40	100.00/100.00	66.26/43.98	53.70/16.70	47.17/30.33		
- Key-step Action Prediction	36.10/17.80	34.40/17.50	32.10/21.50	46.70/18.90	20.12/10.02	42.20/40.70	36.50/17.50	23.40/7.00	38.94/14.02	53.60/34.50	46.60/20.20	83.30/70.00	69.26/42.02	54.60/16.70	41.90/22.57		
- Part State Judgement	45.90/26.70	34.00/15.00	33.00/25.40	54.30/33.30	34.70/18.02	53.30/51.90	41.30/20.80	87.6/7.0	26.28/22.02	53.50/41.60	51.00/34.30	95.00/95.00	49.86/27.98	51.20/16.70	43.67/28.50		
Close-loop Planning Adjustment (Step-wise Success Rate)																	
Proprietary MLLMs																	
GPT-5 [19]	28.84	5.79	38.51	0.00	31.74	12.50	39.43	6.66	41.66	30.55	20.58	10.52	34.14	45.45	29.61		
GPT-5 Mini [19]	14.42	5.17	18.24	0.00	15.78	3.84	25.19	15.00	37.83	9.09	20.58	42.10	0.00	27.27	16.33		
Gemini 2.5 Pro [10]	26.13	13.04	45.62	0.00	42.85	11.53	38.00	5.00	27.02	35.45	23.52	15.78	41.46	31.81	31.73		
Gemini 2.5 Flash [10]	31.06	10.14	34.05	0.00	46.03	19.23	37.90	15.00	24.32	37.03	26.47	42.10	39.02	33.33	31.61		
Open-source MLLMs																	
Qwen3-VL 8B Thinking [21]	23.07	13.04	30.43	0.00	30.15	11.53	36.36	15.00	13.51	37.27	20.58	36.84	0.00	22.72	25.58		
Qwen3-VL 8B Instruct [21]	28.84	24.63	36.95	0.00	38.09	19.23	39.86	10.00	81.00	38.18	23.52	42.10	26.86	4.54	30.65		
Qwen3-VL 235B Thinking [21]	22.11	14.49	33.33	0.00	41.26	23.07	38.56	15.00	35.13	38.18	32.35	42.10	29.26	40.90	31.23		
Qwen3-VL 235B Instruct [21]	25.00	24.63	39.13	0.00	42.85	26.92	35.94	10.00	54.00	28.18	26.47	42.10	24.39	9.09	29.13		
GLM-4.1V-Thinking [26]	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00		
GLM-4.5V [26]	0.00	0.00	0.74	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00		
Embodied Planning Models																	
RoboBrain 2.0-7B [25]	27.27	25.67	34.31	0.00	45.16	53.33	34.86	22.72	5.26	40.77	27.77	36.84	37.03	9.09	31.77		
RoboBrain 2.0-32B [25]	23.14	20.21	28.99	0.00	22.58	13.33	26.31	18.18	7.89	18.44	19.44	26.31	25.92	9.09	21.61		
ApBot [30]	0.80	2.60	4.40	0.00	18.20	25.00	3.70	2.80	5.80	9.90	16.70	26.30	5.90	0.00	7.00		
AppliancePlan (Ours)	31.40	22.97	34.91	0.00	54.84	56.67	42.76	18.18	31.58	63.11	22.22	31.58	25.93	4.55	37.12		
- Manual-App. Part Alignment	34.71	32.43	36.09	0.00	53.23	20.00	43.42	9.09	10.53	47.57	13.89	47.37	37.04	0.00	34.67		
- Key-step Action Prediction	40.50	28.38	36.09	0.00	51.61	53.33	48.68	27.27	21.05	40.78	38.89	42.11	37.04	9.09	38.24		
- Part State Judgement	33.06	27.03	33.73	0.00	56.45	23.33	42.76										



Figure 5: Real-robot execution with AppliancePlan. Representative examples on household appliances.

Appliance Part Grounding. AppliancePlan achieves the best overall grounding performance (22.96% IoU, 22.24% mAP@0.5), outperforming the strongest embodied baseline ApBot by 12.36 IoU and 10.14 mAP@0.5 points with a simpler end-to-end pipeline. The gains stem from explicit manual-conditioned training: without manual context, models cannot reliably associate part descriptions with visual appearance, especially for appliance-specific controls absent from web-crawled data.

Open-loop Manipulation Planning. AppliancePlan achieves 47.86% task completion and 31.36% task success—over 10× the best baseline (4.36%/2.68%)—and ranks first on every appliance category. The near-zero baseline performance confirms that general language and vision capabilities do not transfer to structured multi-step planning with precise parameters; training on (initial-state, goal-state, plan) triples with fully parameterized atomic actions bridges this gap.

Closed-loop Planning Adjustment. AppliancePlan achieves the highest step-wise success rate (37.12%), outperforming the best open-source baseline RoboBrain 2.0-7B by 5.35 points and the best proprietary MLLM Gemini 2.5 Pro by 5.39 points, indicating stronger robustness to execution deviations across appliance categories.

Sequential Planning and Adjustment. This end-to-end setting is the most realistic, as both the initial plan and subsequent corrections are model-generated. AppliancePlan achieves 44.59% completion and 28.07% success, versus 5.84%/4.08% for the best baseline (Qwen3-VL 235B Instruct), confirming that planning and correction gains transfer to long-horizon execution, including recovery from errors introduced by the model’s own predictions.

6.2.3 Ablation of Auxiliary Training Objectives. We ablate the three auxiliary objectives by removing one at a time; results appear in the lower rows of Table 2 (rows marked “-”). Removing **Manual-Appliance Part Alignment** causes the largest grounding degradation, confirming that bidirectional part–manual correspondence is essential for accurate spatial localization. Removing **Key-Step Action Prediction** produces the largest decline on long-horizon metrics—particularly on the realistic sequential setting—indicating that focused supervision on parameter-critical steps is the primary driver of precise action prediction. Removing **Part State Judgment** mainly harms closed-loop recovery, validating its role in enabling state-aware plan revision. Overall, each objective contributes to its targeted capability as designed, and their combination yields the

best balance across grounding, planning, and closed-loop correction.

6.3 Real-Robot Evaluation

To test whether benchmark gains transfer to physical execution, we deploy AppliancePlan on a local workstation with an RTX 4090 GPU and evaluate it on a Franka Emika Panda robot. The system uses an eye-in-hand RealSense D415 camera together with an third-view D415 camera. We test six household appliances: microwave, air fryer, toaster, rice cooker, coffee machine, and blender. For each appliance, we build a CAD/URDF model; FoundationPose [27] uses the CAD model for 6D pose estimation, from which appliance-specific atomic actions are instantiated. For *Pick* actions, we integrate Any-Grasp [8]. During each episode, AppliancePlan first predicts an open-loop plan from the manual, observation, and task instruction before execution, and then revises the next action online using real-time perception feedback. We evaluate 10 manipulation tasks for each appliance. Quantitative results are reported in Table 4, and qualitative examples are shown in Figure 5. As shown in Table 4, AppliancePlan outperforms all baselines on every appliance. Averaged over the six appliances, AppliancePlan achieves 49.87% task completion and 40.00% task success, compared with 19.35%/3.33% for GPT-5 and 9.12%/3.33% for ApBot. The much larger margin on task success indicates that general-purpose MLLMs and pipeline methods may execute early steps correctly, but often fail at the state-dependent decisions required to complete the full task.

Table 4: Real-robot results on household appliances. Entries report task completion rate/task success rate.

	Microwave	Air-fryer	Ricecooker	Coffee Machine	Toaster	Blender
GPT-5	16.67/0.00	11.33/0.00	28.30/10.00	18.30/0.00	14.90/0.00	26.60/10.00
ApBot [30]	11.80/0.00	14.70/0.00	3.30/0.00	8.30/10.00	3.30/0.00	13.30/10.00
AppliancePlan (Ours)	50.00/40.00	49.10/40.00	26.80/20.00	52.50/40.00	33.33/20.00	87.50/80.00

7 Limitations and Future Work

Manual-grounded appliance manipulation involves many sequential steps. Although AppliancePlan improves each individual capability, errors still propagate across stages, leaving considerable room for improving end-to-end task success rates. Future work can build on our data synthesis pipeline, dataset, and end-to-end model by incorporating real-robot reinforcement learning, enabling the model to continuously improve through physical interaction.

References

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Ricardo Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv preprint arXiv:2204.01691* (2022).
- [2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. *NeurIPS* (2022).
- [3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923* (2025).
- [4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. 2024. pi0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164* (2024).
- [5] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Danny Driess, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, et al. 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv preprint arXiv:2307.15818* (2023).
- [6] Open X-Embodiment Collaboration. 2023. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. <https://arxiv.org/abs/2310.08864>.
- [7] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. 2023. PaLM-E: An Embodied Multimodal Language Model. *arXiv preprint arXiv:2303.03378* (2023).
- [8] Hao-Shu Fang, Chenxi Wang, Hongjie Fang, Minghao Gou, Jirong Liu, Hengxu Yan, Wenhai Liu, Yichen Xie, and Cewu Lu. 2023. Anygrasp: Robust and efficient grasp perception in spatial and temporal domains. *IEEE Transactions on Robotics* 39, 5 (2023), 3929–3945.
- [9] Yuzheng Gao, Yuxing Long, Lei Kang, Yuchong Guo, Ziyang Yu, Shangqing Mao, Jiyao Zhang, Ruihai Wu, Dongjiang Li, Hui Shen, and Hao Dong. 2026. RealAppliance: Let High-fidelity Appliance Assets Controllable and Workable as Aligned Real Manuals.
- [10] Google AI. 2025. Gemini 2.5 API. <https://ai.google.dev/gemini-api/docs>
- [11] Yuheng Ji, Huajie Tan, Jiayu Shi, Xiaoshuai Hao, Yuan Zhang, Hengyuan Zhang, Pengwei Wang, Mengdi Zhao, Yao Mu, Pengju An, et al. 2025. Robobrain: A unified brain model for robotic manipulation from abstract to concrete. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 1724–1734.
- [12] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. 2024. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246* (2024).
- [13] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*.
- [14] Xiaoqi Li, Mingxu Zhang, Yiran Geng, Haoran Geng, Yuxing Long, Yan Shen, Renrui Zhang, Jiaming Liu, and Hao Dong. 2024. Manipllm: Embodied multimodal large language model for object-centric robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 18061–18070.
- [15] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. In *NeurIPS*.
- [16] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, et al. 2023. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499* (2023).
- [17] Yuxing Long, Jiyao Zhang, Mingjie Pan, Tianshu Wu, Taewhan Kim, and Hao Dong. 2025. CheckManual: A New Challenge and Benchmark for Manual-based Appliance Manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [18] OpenAI. 2023. Gpt-4 technical report. *arXiv:2303.08774* (2023).
- [19] OpenAI. 2025. Gpt5. <https://openai.com/index/introducing-gpt-5/>
- [20] Mingjie Pan, Jiyao Zhang, Tianshu Wu, Yinghao Zhao, Wenlong Gao, and Hao Dong. 2025. OmniManip: Towards General Robotic Manipulation via Object-Centric Interaction Primitives as Spatial Constraints. *arXiv:2501.03841 [cs.RO]* <https://arxiv.org/abs/2501.03841>
- [21] Qwen Team. 2025. Qwen3-VL API. https://modelstudio.console.alibabacloud.com/?tab=doc#/doc/?type=model&url=2840914_2&modelId=qwen3-vl-plus
- [22] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning*. 8748–8763.
- [23] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3505–3506.
- [24] Team Seedream, Yunpeng Chen, Yu Gao, Lixue Gong, Meng Guo, Qiushan Guo, Zhiyao Guo, Xiaoxia Hou, Weilin Huang, Yixuan Huang, et al. 2025. Seedream 4.0: Toward next-generation multimodal image generation. *arXiv preprint arXiv:2509.20427* (2025).
- [25] BAAI RoboBrain Team, Mingyu Cao, Huajie Tan, Yuheng Ji, Xiansheng Chen, Minglan Lin, Zhiyu Li, Zhou Cao, Pengwei Wang, Enshen Zhou, et al. 2025. Robobrain 2.0 technical report. *arXiv preprint arXiv:2507.02029* (2025).
- [26] GLM-V Team. 2025. GLM-4.5V and GLM-4.1V-Thinking: Towards Versatile Multimodal Reasoning with Scalable Reinforcement Learning. *arXiv:2507.01006 [cs.CV]* <https://arxiv.org/abs/2507.01006>
- [27] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. 2024. Foundationpose: Unified 6d pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 17868–17879.
- [28] Chenfei Wu, Jiahao Li, Jingren Zhou, Junyang Lin, Kaiyuan Gao, Kun Yan, Shengming Yin, Shuai Bai, Xiao Xu, Yilei Chen, et al. 2025. Qwen-image technical report. *arXiv preprint arXiv:2508.02324* (2025).
- [29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
- [30] Jian Zhang, Hanbo Zhang, Anxing Xiao, and David Hsu. 2025. Robot Operation of Home Appliances by Reading User Manuals. *arXiv preprint arXiv:2505.20424* (2025).